SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JAYAKRISHNA.V	**Reg. No.:**	192421259
Section / Batch:	1	**Date:**	27.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I JAYAKRISHNA.V(192421259) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA.V_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Subgraph Isomorphism Problem

1. Problem Understanding

The Subgraph Isomorphism Problem is a classical graph matching problem. Given two graphs, a main graph G and a subgraph H, the objective is to determine whether H can be mapped into G such that all vertices of H are mapped to distinct vertices of G and every edge in H is preserved in G.

A subgraph isomorphism mapping must satisfy the following conditions:

- Every vertex of H is mapped to a unique vertex of G.
- If two vertices are connected by an edge in H, their mapped vertices must also be connected in G.
- The mapping must be one-to-one.
- Extra edges in G between mapped vertices are allowed because H only needs to occur as a subgraph.
- The algorithm should return true if such a mapping exists and false otherwise.

Mathematical Formulation

Let:

G = (V_G, E_G)

H = (V_H, E_H)

where |V_H| <= |V_G|.

A mapping

f : V_H → V_G

is a valid subgraph isomorphism if:

f(u) != f(v) for all u != v

and for every edge:

(u, v) ∈ E_H

we have:

(f(u), f(v)) ∈ E_G

Therefore, the mapping preserves all edges of the smaller graph inside the main graph.

Input and Output Specification

Input: Two graphs represented using adjacency lists.

Example:

```
G = {
   0: [1, 2],
   1: [0, 2, 3],
   2: [0, 1, 3],
   3: [1, 2]
}

H = {
   0: [1],
   1: [0, 2],
```

2: [1]

}

Here, G is the main graph and H is the graph that we want to find inside G.

Output:

Subgraph Isomorphism Exists

Mapping: {0: 0, 1: 1, 2: 2}

This means:

H vertex 0 → G vertex 0

H vertex 1 → G vertex 1

H vertex 2 → G vertex 2

The edges of H are preserved:

0 -- 1

1 -- 2

because the corresponding edges exist in G.

---

2. Constraint Analysis

Constraint checking is used to eliminate invalid vertex mappings while constructing the solution.

Constraint Type	Mathematical Condition	Impact on Search Space
Vertex Uniqueness Constraint	f(u) != f(v) for u != v	Prevents two subgraph vertices from mapping to the same main-graph vertex.
Edge Preservation Constraint	(u,v) ∈ E_H → (f(u),f(v)) ∈ E_G	Rejects mappings that do not preserve required edges.
Vertex Count Constraint	`	V_H
Degree Constraint	degree_G(f(u)) >= degree_H(u)	Prevents mapping a high-degree subgraph vertex to an unsuitable main-graph vertex.
Completion Constraint	All vertices of H are mapped	Terminates the search successfully.

The most important constraints are vertex uniqueness and edge preservation. They ensure that every partial mapping constructed by the algorithm can potentially become a valid subgraph isomorphism.

A degree check can also be applied as an additional pruning condition. If a vertex in H has more neighbors than a candidate vertex in G, that candidate cannot be used.

---

3. State-Space Representation and Pruning Techniques

The search space can be represented as a tree of possible mappings. Each level of the tree corresponds to mapping one vertex of H to a vertex of G.

For the example:

H: G:

0 -- 1 0 -- 1

```
|          | / |
2          2 -- 3
```

The algorithm attempts mappings such as:

H vertex 0 → G vertex 0

H vertex 1 → G vertex 1

H vertex 2 → G vertex 2

giving:

Mapping:

{0: 0, 1: 1, 2: 2}

Branching Decisions

At each recursion level:

- Select an unmapped vertex from H.
- Try every unused vertex from G as its candidate.
- Check whether the candidate satisfies all constraints.
- Add the candidate to the mapping if it is valid.
- Recursively map the next vertex.
- Undo the mapping if the branch cannot produce a solution.

Pruning Rules

The following pruning techniques are applied:

- Used Vertex Pruning: Reject a candidate if it has already been assigned to another vertex of H.
- Edge Constraint Pruning: Reject a candidate if an already mapped neighbor of the current H vertex is not adjacent to the candidate in G.
- Degree Pruning: Reject candidates whose degree in G is smaller than the degree required by the corresponding vertex in H.
- Vertex Count Pruning: If H contains more vertices than G, immediately return false.
- Early Success: Stop when all vertices of H have been mapped.
- Backtracking: Remove the latest mapping when the current branch fails.

These pruning rules reduce the number of mappings that need to be explored.

---

4. Pseudocode Using Backtracking

INPUT:

  Main graph G

  Subgraph H

OUTPUT:

  true if H is isomorphic to a subgraph of G

  false otherwise

```
mapping = empty map
```

```
used = empty set

FUNCTION isSafe(hVertex, gVertex):

    IF gVertex is already in used:
        RETURN false

    IF degree(G, gVertex) < degree(H, hVertex):
        RETURN false

    FOR each neighbor hNeighbor of hVertex:

        IF hNeighbor is already in mapping:

            gNeighbor = mapping[hNeighbor]

            IF gNeighbor is not adjacent to gVertex in G:
                RETURN false

    RETURN true


FUNCTION backtrack(position):

    IF position == number of vertices in H:
        RETURN true

    hVertex = next unmapped vertex in H

    FOR each gVertex in G:

        IF isSafe(hVertex, gVertex):

            mapping[hVertex] = gVertex
            add gVertex to used

            IF backtrack(position + 1):
                RETURN true
```

```
        remove gVertex from used
        remove hVertex from mapping

    RETURN false



FUNCTION subgraphIsomorphism(G, H):

    IF number of vertices in H > number of vertices in G:
        RETURN false

    mapping = empty map
    used = empty set

    IF backtrack(0):
        RETURN true
    ELSE:
        RETURN false



MAIN:

    IF subgraphIsomorphism(G, H):
        PRINT "Subgraph Isomorphism Exists"
        PRINT "Mapping:", mapping
    ELSE:
        PRINT "No Subgraph Isomorphism Exists"
```

Important Functions

isSafe(hVertex, gVertex)

This function verifies whether a vertex of H can safely be mapped to a candidate vertex of G.

It checks:

1. Whether the candidate vertex is already used.
2. Whether the candidate has sufficient degree.
3. Whether all already mapped neighbors of hVertex have corresponding edges in G.

backtrack(position)

This recursive function:

1. Selects an unmapped vertex from H.
2. Tries each unused vertex from G.
3. Applies the constraint checks.

4. Adds a valid mapping.
5. Recursively continues the search.
6. Removes the mapping if the branch fails.

---

5. Execution and Testing

Sample Input Graph

The graph shown in the program can be represented as:

G = {
  0: [1, 2],
  1: [0, 2, 3],
  2: [0, 1, 3],
  3: [1, 2]
}

H = {
  0: [1],
  1: [0, 2],
  2: [1]
}

The main graph contains the edges:

0 -- 1

0 -- 2

1 -- 2

1 -- 3

2 -- 3

The subgraph contains:

0 -- 1

1 -- 2

Execution Trace

The backtracking algorithm starts by selecting vertex 0 from H.

Step 1: Map H vertex 0

Try:

H vertex 0 → G vertex 0

This is valid.

Mapping = {0: 0}

Used = {0}

Step 2: Map H vertex 1

Vertex 1 in H is adjacent to vertex 0.

Try:

H vertex 1 → G vertex 1

Since G contains the edge:

0 -- 1

the mapping is valid.

Mapping = {0: 0, 1: 1}

Used = {0, 1}

Step 3: Map H vertex 2

Vertex 2 in H is adjacent to vertex 1.

Try:

H vertex 2 → G vertex 2

The edge:

1 -- 2

exists in G.

Therefore:

Mapping = {0: 0, 1: 1, 2: 2}

All vertices of H have been mapped.

Output

Subgraph Isomorphism Exists

Mapping: {0: 0, 1: 1, 2: 2}

Mapping Verification

The mapping is:

H vertex 0 → G vertex 0

H vertex 1 → G vertex 1

H vertex 2 → G vertex 2

The edges of H are preserved:

H: 0 -- 1

G: 0 -- 1

H: 1 -- 2

G: 1 -- 2

Therefore, H occurs as a subgraph of G.

---

6. Correctness Justification

The algorithm is correct because it systematically explores possible one-to-one mappings from the vertices of H to vertices of G.

At every step, a candidate mapping is accepted only when:

- The target vertex in G has not already been used.
- The target vertex has sufficient degree.
- Every edge from the current H vertex to an already mapped vertex is preserved in G.

Therefore, every partial mapping maintained by the algorithm satisfies the required constraints.

If the recursion reaches:

position == |V_H|

then every vertex of H has been assigned a distinct vertex of G, and all required edges have been preserved. Hence, the mapping is a valid subgraph isomorphism.

If a particular mapping cannot be completed, the algorithm backtracks and tries another unused vertex of G.

Because the algorithm considers all possible valid candidates and only prunes mappings that violate necessary constraints, it will find a valid mapping whenever one exists.

Thus, the algorithm is both complete and correct.

---

7. Complexity Analysis

Time Complexity

Suppose:

|V_G| = n

|V_H| = m

with:

m <= n

The algorithm may try different one-to-one mappings of the m vertices of H to the n vertices of G.

The number of possible mappings is:

n × (n - 1) × (n - 2) × ... × (n - m + 1)

which is:

n! / (n - m)!

Therefore, the worst-case time complexity is approximately:

O(n! / (n-m)!)

For the special case where m = n, this becomes:

O(n!)

Additional adjacency and constraint checks can increase the polynomial factor, but the factorial component dominates the complexity.

Best-Case Complexity

The best case occurs when a valid mapping is found immediately.

If the first candidate at every level satisfies the constraints, the search can finish after mapping all m vertices:

O(m)

apart from the cost of the individual constraint checks.

Auxiliary Space Complexity

The algorithm stores:

- The mapping of m vertices.
- The set of used vertices.
- A recursion stack of depth at most m.

Therefore, the auxiliary space required by the backtracking procedure is:

O(m)

The adjacency-list representation of the graphs requires additional space depending on the number of edges:

O(|V_G| + |E_G| + |V_H| + |E_H|)

---

8. Complexity Class Categorization

The Subgraph Isomorphism Problem belongs to the class NP because a proposed mapping can be verified in polynomial time.

To verify a candidate mapping:

1. Check that every vertex of H has been assigned.
2. Check that no two vertices of H map to the same vertex of G.
3. Check every edge of H.
4. Confirm that the corresponding edge exists in G.

These verification steps can be performed in polynomial time.

The decision version of the Subgraph Isomorphism Problem is NP-Complete.

It is NP-Hard because several well-known NP-Complete problems can be reduced to it. For example, the Hamiltonian Path Problem can be expressed as a special case of subgraph isomorphism by asking whether a path graph with n vertices occurs as a subgraph of a given graph with n vertices.

Although the problem is computationally difficult for large graphs, backtracking provides a complete solution for small and moderately sized graphs, especially when effective pruning constraints are applied.

---

9. Advantages and Limitations

Advantages

- Easy to understand and implement.
- Guarantees a correct result when all candidate mappings are explored.
- Uses recursion naturally to explore the mapping search space.
- Constraint checking eliminates many invalid mappings early.
- Degree-based pruning can improve performance.
- Can terminate immediately when a valid subgraph mapping is found.
- Works with adjacency lists or adjacency matrices.

Limitations

- Worst-case running time is factorial.
- The number of possible vertex mappings increases rapidly as the graphs become larger.
- Backtracking can still explore a very large search tree.
- Performance depends heavily on the ordering of vertices and the effectiveness of pruning.
- It is not suitable for very large graphs without additional optimization techniques.

Conclusion

The backtracking approach provides a complete method for solving the Subgraph Isomorphism Problem. It constructs a mapping between the subgraph and the main graph one vertex at a time. By applying vertex uniqueness, edge preservation, degree constraints, and backtracking, invalid mappings are rejected early while valid mappings are preserved.

For the given example, the algorithm successfully finds:

Mapping: {0: 0, 1: 1, 2: 2}

and produces:

Subgraph Isomorphism Exists

Thus, the given graph H is present as a subgraph of G.

OUTPUT